

BULUT MİMARİLERİNDE TEST MÜHENDİSLİĞİ

RESİM BOYUTLANDIRMA MİKROSERVİSİ FİNAL PROJE RAPORU

Asenkron FastAPI, Pillow, PostgreSQL, LocalStack S3 ve Kapsamlı Test
Otomasyonu

Hazırlayan: Arif Küçükeşmekaya
Danışman: Büşra Ayaksız
Okul: Marmara Üniversitesi
Ders: Bulut Mimarilerinde Test Mühendisliği (Dönem Projesi)
Tarih: Haziran 2026

1. Giriş ve Yönetici Özeti

Modern bulut tabanlı uygulamalarda, kullanıcılar tarafından yüklenen medya dosyalarının işlenmesi, optimizasyonu ve depolanması en kritik mimari bileşenlerden biridir. Bu proje kapsamında, FastAPI tabanlı, asenkron çalışabilen, yüksek performanslı ve bulut-native standartlara sahip bir Resim Boyutlandırma Mikroservisi (Image Resize Service) geliştirilmiştir. Projenin ana odağı, sadece işlevsel bir kod yazmak değil; yazılımın güvenilirliğini, performansını, stres altındaki davranışını ve güvenliğini test mühendisliği prensipleriyle doğrulamaktır.

1.1. Mimari Tasarım

Uygulama, mikroservis ve bulut mimarisi standartlarına uygun olarak tasarlanmıştır. İstemci, boyutlandırmak istediği görseli ve hedef boyutları HTTP POST metodu ile gönderir. Sunucu, Pillow resim işleme kütüphanesi yardımıyla görseli bellekte işler. Ardından, AWS S3 API'sini yerel ortamda simüle eden LocalStack servisine yükler. İşleme ait metadata kayıtları ise PostgreSQL veritabanında saklanır. Kullanıcıya doğrudan dosya adresi verilmek yerine, güvenlik amacıyla belirli bir süre geçerli olan (Presigned URL) geçici indirme linkleri üretilir.

1.2. Kullanılan Başlıca Teknolojiler

- FastAPI: Asenkron API yönlendirme ve middleware katmanı.
- Pillow: Performanslı ve kayıpsız resim ölçekleme motoru.
- PostgreSQL & SQLAlchemy: Kalıcı veri depolama ve ORM katmanı.
- LocalStack & Moto: AWS S3 entegrasyonu ve test mocks altyapısı.

2. Bulut Depolama ve Veri Tabanı Tasarımı

Sistemin veri katmanı, dosya depolama ve ilişkisel metadata depolama olmak üzere iki ana kısma ayrılmıştır. Bu tasarım, nesne depolama (Object Storage) ve ilişkisel veritabanlarının (RDBMS) güçlü yönlerini bir araya getirir.

2.1. AWS S3 Depolama Yapılandırması

Yüklenen resimlerin doğrudan sunucu diskinde saklanması, sunucu doluluğu ve ölçeklenebilirlik açısından tehlikelidir. Bu yüzden dosyalar S3 uyumlu nesne depolama servisinde saklanır. Geliştirme ortamında bu servis LocalStack S3 konteyneri ile simüle edilir. Sunucu, her resme rastgele bir UUID atayarak S3'e kaydeder. İndirme talepleri için ise kovanın (bucket) erişim politikası kapalı tutularak, boto3 SDK ile 1 saat geçerli Presigned URL'ler oluşturulur.

2.2. Veritabanı Şeması ve ORM Yapısı

SQLAlchemy ORM kullanılarak tanımlanan ImageRecord tablosu, resimlere ait tüm teknik bilgileri saklar. Tablo alanları şu şekildedir:

Kolon Adı	Veri Tipi	Açıklama
id	Integer (PK)	Tablonun birincil anahtarı (Auto-increment)
original_filename	String	Kullanıcının yüklediği dosyanın orijinal adı
s3_bucket	String	Dosyanın yüklendiği S3 kova ismi
s3_key	String	Resmin S3 üzerindeki UUID formatlı benzersiz yolu
resized_width	Integer	Boyutlandırılmış görselin genişliği (piksel)
resized_height	Integer	Boyutlandırılmış görselin yüksekliği (piksel)
file_size_bytes	Integer	Boyutlandırılmış görselin disk boyutu (byte)
created_at	DateTime	Kayıt oluşturulma zaman damgası (UTC)

3. Kapsamlı Test Otomasyonu ve Kalite Güvencesi

Projenin güvenilirliğini ve bulut standartlarına uygunluğunu doğrulamak amacıyla üç aşamalı bir test stratejisi kurgulanmıştır. Otomatik test kapsamımız (Coverage) %80.55 seviyesindedir.

3.1. Birim Testler (Unit Tests)

Birim testlerinde Pillow boyutlandırma algoritmaları ve Pydantic girdi sınırları test edilmiştir. Negatif boyutlar, desteklenmeyen uzantılar veya hatalı byte'lar içeren resimlerin sunucu tarafından doğru hatalarla engellendiği dış servislere bağlanılmadan doğrulanmıştır.

3.2. Entegrasyon Testleri (Integration Tests)

Entegrasyon testlerinde veritabanı (in-memory SQLite) ve S3 depolama işlemleri test edilmiştir. AWS S3 testleri için Moto (mock_aws) kütüphanesi kullanılmıştır. Moto, AWS istemci çağrılarını bellek içinde yakalayıp yapay bir S3 sunar. Böylece testler internet bağımsız ve hızlı çalışır. API endpoints testleri ise FastAPI TestClient ile gerçekleştirilmiştir.

3.3. Uçtan Uca Testler (E2E Tests - Playwright)

Playwright otomasyon aracı ile gerçek Chromium tarayıcısı üzerinde kullanıcı senaryoları simüle edilmiştir. Resim yükleme, parametreleri girme, listeyi yenileme ve silme adımları otomatik olarak oluşturulmuştur. Testlerin izolasyonunu sağlamak için her test başlangıcında SQLite test veritabanı sıfırlanmaktadır.

3.4. Test Kapsama (Coverage) Analizi

Pytest-cov raporumuza göre %80.55'lik kapsama ulaşılmıştır. Kapsanmayan %19'luk kod parçaları incelendiğinde, bunların try/except bloklarındaki ekstrem hata loglama satırları olduğu doğrulanmıştır. Bu durum test mühendisliği standartlarında tamamen kabul edilebilir bir düzeydir.

4. Performans Mühendisliği ve Gözlemlenebilirlik

Bir bulut mikroservisinin yoğun yük altındaki davranışını ölçmek ve izlemek, test mühendisliğinin temel parçalarından biridir.

4.1. Grafana k6 ile Yük ve Stres Testleri

Grafana k6 aracı kullanılarak sisteme yoğun yük bindirilmiştir. 50 sanal kullanıcı (VUs) ile 30 saniye boyunca eşzamanlı istekler gönderilmiştir. Test sonucunda saniyede ortalama 50 istek (RPS) başarıyla işlenmiştir. Performans barajı (threshold) olarak belirlediğimiz p95 gecikme süresi < 2sn ve hata oranı < %5 limitleri başarıyla aşılmıştır.

4.2. Prometheus Metrik Toplama Altyapısı

FastAPI uygulamamız bünyesindeki `/metrics` endpoint'i yardımıyla Prometheus'a metrik sağlar. Toplanan özel metrikler şunlardır:

- `http_requests_total`: Gelen tüm HTTP isteklerinin method, path ve status bazlı sayısı.
- `http_request_duration_seconds`: İsteklerin yanıt sürelerinin histogram dağılımı.
- `resize_operations_total`: Resim boyutlandırma işlemlerinin format ve başarı kırılımı.

4.3. Grafana Dashboard Yapılandırması

Grafana container'ı ayağa kalktığında Prometheus veri kaynağı ve hazırladığımız dashboard şablonu otomatik olarak yüklenir. Bu dashboard sayesinde sunum esnasında k6 testi çalıştırılırken grafiklerin (Request Rate, Latency, Ops Rate) anlık değişimini izlemek mümkün olmaktadır. Bu kavram sistemin Gözlemlenebilirliğini (Observability) doğrulamaktadır.

5. Güvenlik Sıkılaştırması (Security Hardening)

Proje geliştirilirken Secure Coding (Güvenli Kodlama) kurallarına sıkı sıkıya uyulmuştur:

1. DDoS ve RAM Şişmesi Koruması: Büyük resimlerin sunucu RAM'ini kilitlememesi için ASGI middleware katmanında 10MB dosya sınırı uygulanmış ve bu sınırı aşan istekler belleğe alınmadan HTTP 413 Payload Too Large hatasıyla reddedilmiştir.
2. Path Traversal & S3 Key Injection Koruması: Kullanıcının gönderdiği dosya isimleri doğrudan diskte veya S3'te kullanılmamaktadır. Dosya adları rastgele UUID'ler ile maskelenerek `../../etc/passwd` gibi dizin aşımı saldırıları engellenmiştir.
3. XSS (Cross-Site Scripting) Koruması: Arayüzde (HTML/JS) API'den dönen veriler ekrana yazılırken kesinlikle innerHTML kullanılmamış, tarayıcının güvenli textContent ve createElement metotları tercih edilmiştir.
4. Veri Erişim Gizliliği: S3 bucket tamamen kapalı tutulmuş, resimlerin indirilmesi için 1 saat süreli Presigned URL'ler üretilmiştir.

6. Sonuç ve Değerlendirme

Bu proje kapsamında, bulut tabanlı bir resim boyutlandırma mikroservisi en yüksek test mühendisliği standartlarıyla geliştirilmiştir. Projede uygulanan test piramidi, otomatik mocks altyapısı, dockerize yük testi senaryoları, otomatik konfigüre edilen metrik panelleri ve Kubernetes manifestleri; uygulamanın gerçek bulut ortamlarına dağıtım ve ölçeklenmeye tam uyumlu olduğunu kanıtlamaktadır.

Proje Sahibi:

Arif Küçükeşmekaya

Danışman:

Büşra Ayaksız